



Mise en place d'un Catalogue Distribué pour Marketplace

Réalisé par :

Khallou Yousra
Ait Oumakhir Ilham
Zakraoui Meryem

Encadré par :

Mme .Yasmina El Morabit



Sommaire

1

Introduction

2

Architecture et Mise en Place

3

Ingestion des Données

4

Visualisation et Monitoring

5

Traitement et Optimisation

6

Conclusion et Perspectives



Contexte du projet

Un site e-commerce de type marketplace héberge des millions de produits. Pour gérer la croissance, le système doit répartir les données entre plusieurs shards tout en assurant une recherche rapide et une disponibilité élevée.



Architecture et Mise en Place



Définitions :

le **sharding** est une méthode qui permet de partitionner un ensemble de données venant d'une même base de données.

Il existe deux principales méthodes de sharding ou partitionnement : le **partitionnement horizontal** et le **partitionnement vertical**.

Le **partitionnement horizontal** permet de séparer les lignes et de les stocker dans différents serveurs.

Le **partitionnement vertical** quant à lui permet de séparer les colonnes et de les stocker dans différents serveurs.

Concept	Explication
Shard	Un nœud ou un replicaset qui contient une partie des données.
Chunk	Plage de valeurs du shard key (ex : catégories 1–1000).
Shard Key	La clé utilisée pour répartir les documents → ex : categorie_id , vendors_id
Config Server	Stocke la méta-donnée du cluster (quels chunks dans quels shards).
Mongos Router	Reçoit les requêtes et les route vers le bon shard.
ReplicaSet	un groupe de serveurs qui contiennent la même donnée (Primary + Secondaries).

Composition d'un ReplicaSet MongoDB

Élément	Rôle principal	Caractéristiques	Participe aux élections	Stocke des données
Primary	Gérer les écritures + piloter la réplique	Reçoit toutes les écritures, envoie les opérations aux Secondaries	✓ Oui	✓ Oui
Secondary	Haute disponibilité + lectures	Réplique en continu l'oplog du Primary, peut devenir Primary	✓ Oui	✓ Oui
Arbiter	Participer aux votes	Aucun stockage, utilisé pour éviter un nombre pair de nœuds	✓ Oui	✗ Non

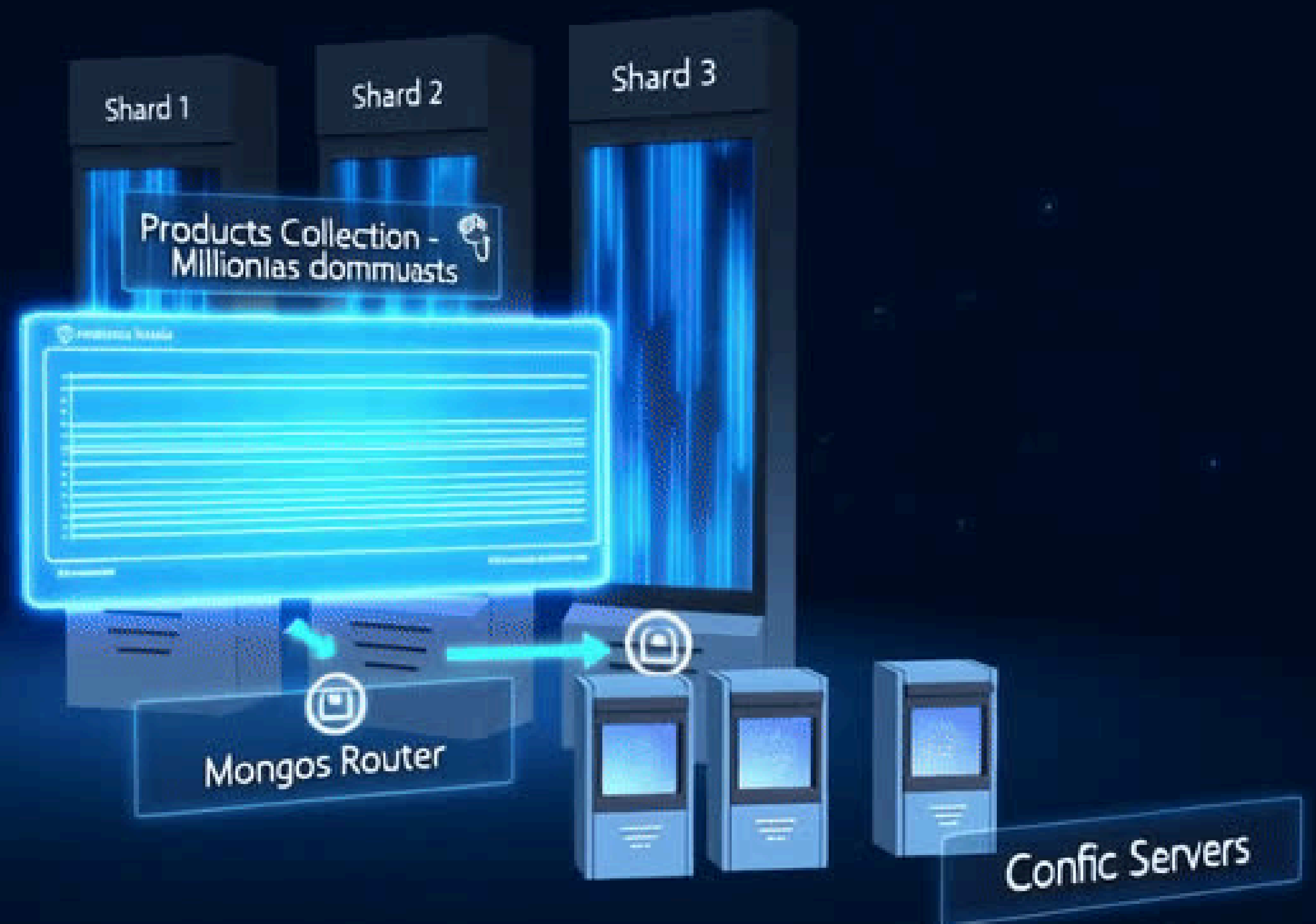


Avantages :

Répartition de
charges entre
plusieurs serveurs

**Recherches plus
rapides
Scalabilité horizontale**

Haute disponibilité
via replica sets



Déploiement Docker de shards et config

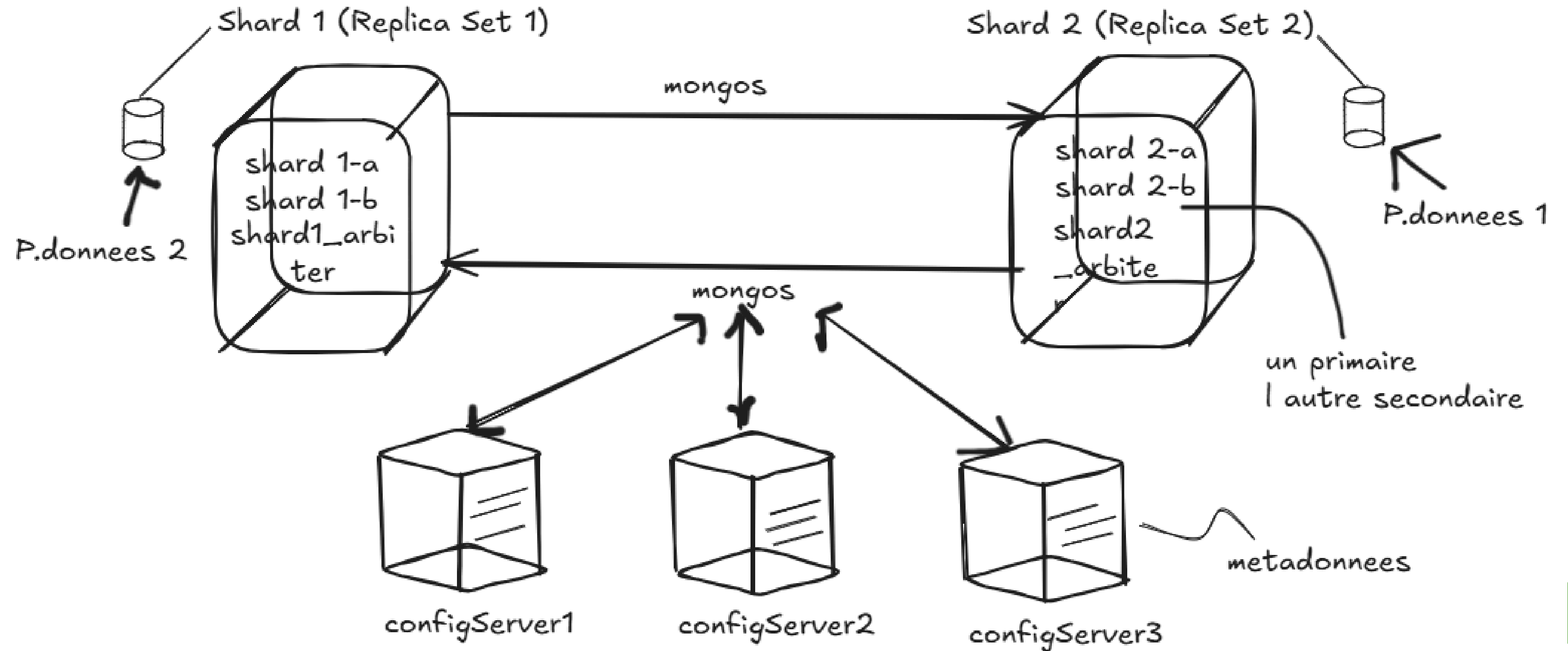
servers :



27020:27020			
	configsvr2 ●		
	mongo:4.4		
	27029:27019		
	configsvr3 ●		
	mongo:4.4		
	27039:27019		
	shard1b ●		
	mongo:4.4		
	27028:27018		
	shard1_arbiter ●		
	mongo:4.4		
	27040:27018		
	shard2_arbiter ●		
	mongo:4.4		
	27041:27020		

	mongos ●		
	mongo:4.4		
	27017:27017		
	configsvr1 ●		
	mongo:4.4		
	27019:27019		
	shard1a ●		
	mongo:4.4		
	27018:27018		
	shard2b ●		
	mongo:4.4		
	27030:27020		
	shard2a ●		
	mongo:4.4		
	27020:27020		
	configsvr2 ●		
	mongo:4.4		
	27029:27019		

Explication de déploiement Docker :



test de failover - la panne :

```
MongoDB server version: 4.4.29  
shard1a:27018 - PRIMARY  
shard1b:27018 - SECONDARY  
shard1_arbiter:27018 - ARBITER
```



```
C:\Users\hp\Desktop\mongo-sharded>docker stop shard1a  
shard1a
```



```
MongoDB server version: 4.4.29  
shard1a:27018 - SECONDARY  
shard1b:27018 - PRIMARY  
shard1_arbiter:27018 - ARBITER
```

```
MongoDB server version: 4.4.29  
shard1a:27018 - (not reachable/healthy)  
shard1b:27018 - PRIMARY  
shard1_arbiter:27018 - ARBITER
```



```
C:\Users\hp\Desktop\mongo-sharded>docker start shard1a  
shard1a
```



Ingestion des Données

- 1 Définition
- 2 Objectifs
- 3 Types
- 4 Sources de Données
- 5 Architecture Moderne
- 6 Concepts Clés
- 7 Pourquoi l'Ingestion ?

Objectifs de l'Ingestion

1

Centraliser les données
provenant de sources multiples

2

Automatiser l'arrivée continue
des données

3

Garantir la qualité et la
cohérence

4

Assurer la scalabilité (supporter
un volume important)

5

Minimiser la latence lorsqu'il
s'agit de données temps réel

6

Alimenter les systèmes d'analyse

Types d'Ingestion

Type d'ingestion	Avantages	Inconvénients
Batch Ingestion	- Simple à implémenter- Efficace pour gros volumes- Peu coûteux	- Latence élevée- Pas adapté au temps réel
Streaming / Real-Time	- Faible latence- Idéal pour IoT et monitoring - Adapté aux systèmes distribués	- Plus complexe- Nécessite un message broker
Lambda / Hybrid	- Temps réel + traitement massif- Approche robuste pour production	- Architecture plus complexe

Sources de Données pour l'Ingestion

Sources internes

- Bases SQL / NoSQL
- Fichiers internes (CSV, XML, JSON)
- API internes
- Applications métiers

Sources externes

- APIs publiques (REST, GraphQL)
- Sites web (scraping autorisé)
- Plateformes e-commerce (ex. Etsy, eBay, Amazon)
- Réseaux sociaux
- Streaming IoT (capteurs, devices...)

Sources semi-structurées

- CSV, JSON, XML
- Streams d'événements
- Logs (ELK, Logstash...)

Architecture moderne d'ingestion

Producteurs (Producers)

- Collecter les données depuis une source
- Nettoyer/valider minimalement
- Envoyer la donnée brute vers un broker



Message Broker

- Absorber un fort volume de messages
- Garantir la livraison (at-least-once, exactly-once...)
- Éviter que les consumers soient surchargés
- Gérer l'ordonnancement et la répartition

Consumers

- Lire les messages du broker
- Nettoyer / valider / enrichir les données
- Insérer dans une base de données
- Gérer les erreurs
- Scaler horizontalement

Système de stockage distribué

- Scalabilité horizontale
- Tolérance aux pannes
- Haute disponibilité
- Performance élevée
- Gestion de très grands volumes de données

-  Ask Gordon BETA
-  Containers
-  Images
-  Volumes
-  Builds
-  Models BETA
-  Docker Hub
-  Docker Scout
-  Extensions

Containers [Give feedback](#)

View all your running containers and applications. [Learn more](#)

Container CPU usage ⓘ
30.37% / 800% (8 CPUs available)

Container memory usage ⓘ
1.74GB / 3.63GB

Show charts



☒ Only show running containers

☐		Name	Container ID	Image	Port(s)	CPU (%)	Last sta	Actions
☐	▼	mongo-sharded	-	-	-	30.37%	2 minut	 ⋮ 
☐		consumer	b6096d78c2d1	mongo-sha		0.94%	2 minut	 ⋮ 
☐		producer	1340a25ce2f6	mongo-sha		0.38%	5 minut	 ⋮ 
☐		mongos	4b43822e5166	mongo:6.0	27025:27017 ↗	0.89%	6 minut	 ⋮ 
☐		rabbitmq	bcbfb1bd4b8c	rabbitmq:3	15672:15672 ↗ Show all ports (2)	2.86%	6 minut	 ⋮ 

Showing 22 items

Concepts Clés dans l'Ingestion

● **Scalabilité:**

la capacité du système Pouvoir gérer plus de données en ajoutant des ressources.

● **Tolérance aux pannes:**

même si une partie du système tombe en panne, les données ne sont pas perdues.

● **Déduplication:**

éviter d'enregistrer plusieurs fois la même donnée.

● **Idempotence:**

même si on traite plusieurs fois la même donnée, le résultat final reste le même.

● **Durabilité:**

Garantir que les données sont stockées même en cas de crash.



Pourquoi l'Ingestion est cruciale dans ce projet ?

- **RabbitMQ** assure la file d'attente durable
- Le Producer alimente le système sans bloquer
- Le Consumer traite les données en continu
- **MongoDB** Shardé permet de scaler horizontalement l'ingestion
- Le shard key **seller_id** répartit les données entre les shards
- Tolérance aux pannes → aucun produit n'est perdu



**Visualisation
et Monitoring
+
Traitement et
Optimisation**

Dashboard Streamlit + Analyse de Performance

c'est quoi Streamlit ?

Streamlit est un framework Python qui permet de créer des dashboards et des interfaces web interactives très facilement.

Il suffit d'écrire du code Python, et Streamlit génère automatiquement un site web moderne



Qu'est-ce qu'un index en MongoDB ?

Un index est une structure de données que MongoDB crée pour rechercher des documents plus rapidement.

- **Sans index** → MongoDB doit scanner toute la collection **(COLLSCAN)**.
- **Avec index** → MongoDB cherche directement dans l'arbre indexé **(IXSCAN)**.

Un index est comme l'index d'un livre : au lieu de lire toutes les pages, on va directement à la bonne section.

Type d'index	Fonction	Exemple	Importance
● Simple	Filtre rapide	{ category_id: 1 }	• Accélère les filtres simples • Réduit totalDocsExamined • Très léger • Très utilisé dans les champs fréquents
● Composite	Multicritères / tri	{ category_id: 1, price: 1 }	• Recherche sur plusieurs critères • Accélère tri + filtre • Très performant pour les requêtes multi-conditions
● Texte	Recherche full-text	{ title: "text" }	• Recherche “Google-like” • Accélère les recherches de mots • Obligatoire pour les systèmes de recherche utilisateur
● Partiel	Index ciblé	partialFilterExpression	• Moins lourd • Plus rapide • Index hyper ciblé • Idéal pour les champs peu utilisés ou filtrés
● Unique	Unicité	{ email: 1 }	Garantit l'unicité : • emails • usernames • SKU produits • vendor_id si nécessaire

> category_id_1	REGULAR ⓘ	880.6 kB	0 (since Thu Dec 04 2025)	
> vendor_id_1	REGULAR ⓘ	1.0 MB	0 (since Thu Dec 04 2025)	SHARD KEY ⓘ
> category_id_1_price_1	REGULAR ⓘ	1.4 MB	0 (since Thu Dec 04 2025)	COMPOUND ⓘ
> category_id_1_attributes.color_1_attributes.size_1_rating_avg_-1	REGULAR ⓘ	892.9 kB	0 (since Thu Dec 04 2025)	COMPOUND ⓘ
> title_text_description_text_brand_text	TEXT ⓘ	25.3 MB	0 (since Thu Dec 04 2025)	COMPOUND ⓘ
> seller_id_1_status_1_updated_at_-1	REGULAR ⓘ	995.3 kB	0 (since Thu Dec 04 2025)	COMPOUND ⓘ
> category_id_1_status_1_stock.quantity_-1	REGULAR ⓘ	24.6 kB	0 (since Thu Dec 04 2025)	PARTIAL ⓘ COMPOUND ⓘ

Pourquoi analyser ces requêtes ?

Dans un système e-commerce/marketplace, les utilisateurs exécutent toujours les mêmes types de requêtes :

- Filtrer
- Rechercher
- Trier
- Agréger
- Explorer par catégorie ou vendeur

Ces opérations sont critiques pour un vrai site.

Elles permettent donc de mesurer la qualité des indexes, la rapidité du sharding, et les performances globales de la base.

Les 6 Requêtes utilisées — Et pourquoi elles sont essentielles



Requête simple (Filtrage par catégorie)

- C'est la requête la plus fréquente en e-commerce
- Permet de tester un index simple
- Vérifie si MongoDB utilise l'index ou fait un COLLSCAN
- Sert de baseline pour toutes les autres requêtes



Requête composée (Filtrage multicritère)

- Cas très fréquent : filtrage par catégorie + prix
- Permet d'évaluer un index composite
- Montre l'impact exact du choix de l'ordre des champs dans un index



Requête de tri (Sorting)

- Le tri est extrêmement coûteux sans index
- Test essentiel pour vérifier si l'index est utilisé pour le tri

Les 6 Requêtes utilisées — Et pourquoi elles sont essentielles



Agrégation : Prix moyen par catégorie

- Teste la puissance du pipeline d'agrégation
- Requête très lourde → stress test CPU
- Montre comment MongoDB agrège sur plusieurs shards



Agrégation : Nombre de produits par vendeur

- Très utilisée pour les dashboards internes
- Très bonne requête pour tester la clé de sharding (vendor_id)



Recherche textuelle (regex recherche produit)

- Reproduit la recherche utilisateur : "laptop", "iphone"...
- Test final pour valider l'index texte



**Merci pour
votre attention**